SimpleSnake: Evaluating 2D Text-based Spatial Reasoning in LLMs

Porter Rigby
porterrigby@u.boisestate.edu

Abstract

LLMs have come to be used for a wide range of tasks outside of their originally intended domains. These models are inherently designed to create associations between words within their context windows, but their ability to make associations between non-word entities and text-based space is not fully explored. This paper proposes a new game for the Clembench framework called SimpleSnake, aimed at evaluating LLMs on their aptitude for performing 2D text-based spatial reasoning inspired by the classic game of snake, and shows potential for navigating such spaces using large logic-oriented models.

1 Introduction

Large language models (LLMs) have come to be used in recent years for a wide variety of tasks, from driving chatbot applications to performing data classification and labeling. Although initially intended for tasks like language translation, it is now evident that LLMs are becoming increasingly capable of performing tasks tangential to their intended domains. As a result, there are interesting questions that arise surrounding what types of tangential tasks these models are capable of performing, and how well they can perform them.

LLMs are obviously capable of making associations between words in a context window, based on their foundational architecture (Vaswani et al., 2023). However, something that is not clear is whether LLMs are capable of making associations between points in n-dimensional spaces and intelligently navigating those spaces without being specifically trained to do so. This paper proposes SimpleSnake, an evaluation task based on the Clembench [1] LLM evaluation framework (Chalamalasetti et al., 2023) to bridge this gap and test a model's aptitude for 2D text-based spatial reasoning. This evaluation is accomplished by engaging a model in variations of the classic game Snake, to test different aspects of understanding and performance.

To date, the Clembench framework implements two games in particular (AdventureGame and ReferenceGame) that are similarly aligned with the tasks presented in SimpleSnake. These games test a model's ability to act coherently and intelligently on it's own previous actions, as well as identifying ASCII art based on referring expressions respectively. While both contain some overlap with the objectives in the proposed SimpleSnake game, they are disjoint from each other and do not evaluate a model's ability to navigate an evolving 2D space. Because of this, SimpleSnake is put forward as an additional game for benchmarking models within the framework.

The rest of this paper is as follows. First, I explain some of the background and related work relevant to SimpleSnake. Second, I discuss the overarching implementation of SimpleSnake as a game within Clembench. Third, is an explanation of the different game variations implemented within SimpleSnake, along with accompanying metrics and results from evaluating a selection of models. Finally, I discuss the potential implications that arise from this work and possible subsequent research directions.

2 Background and Related Work

2.1 Background

The game of Snake is arguably one of the most well-known classic digital games to have been created. Growing from early arcade games, it came pre-installed on Nokia devices [2] in the late 1990s, giving it incredibly high visibility and raising awareness of its existence. The game itself comes in many forms and has been the basis for multiple derivative works.

[1] https://clembench.github.io/

[2] https://www.nokia.com/

Snake is typically played on a two dimensional square grid, with a handful of rules common to all variations. A player controls a line-like entity considered a 'snake' within the grid, which moves forward in single-cell steps based on specified directions. If the snake leaves the grid space or runs into itself, the game ends. In addition to the snake, other entities known as 'food' spawn within the grid, with the goal of the game being for the player to navigate the snake towards the location of the food. As the player successfully navigates the snake to a food entity, the food is consumed and the snake grows in length, adding to the challenge of keeping the snake from colliding with itself or the grid boundaries. Typically, a player's success is gauged by the number of times the snake is successfully able to 'consume' the food.

Various different versions of the game exist, adding twists to the rules and gameplay to create further challenge. Some of these variations include adding scaling movement speed to the snake, ways for the snake to teleport around the grid, and adding obstacles that must be avoided. Outside of the base game, SimpleSnake takes inspiration from these twists and implements a variation with obstacles, as well as a variation based on route planning.

2.2 Related Work

Chatbot Arena Chatbot Arena (formerly LMSYS) is an open platform for providing human-evaluations of LLMs based on a preference-centric ranking system (Chiang et al., 2024). Many of the primary use-cases for generative language models revolve around producing quality dialogue, which is inherently difficult to systematically evaluate. Chatbot Arena approaches this by collecting human feedback for model responses and using this feedback to rank models by overall human preference. SimpleSnake works within the Clembench framework, which itself builds upon the type of interactive dialogue present in Chatbot Arena.

Clembench Clembench is a framework for the systematic evaluation of language models on interactive dialogue tasks (Chalamalasetti et al., 2023). The framework combines some of the benefits of evaluating on interactive dialogue tasks like are present in Chatbot Arena (Chiang et al., 2024), with the benefits of more traditional, systematic, and repeatable benchmarks. This is accomplished by making models play games under controlled circumstances, recording their interactions, and scoring them based on their ability to adhere to game rules and their qualitative performance on each game. Clembench is highly extensible, and encourages community prototyping and development of new games to test models of various sizes. The benchmark integrates API access for many of the most popular and capable LLMs, allowing for easy and relevant gameplay evaluation. SimpleSnake directly embraces this extensibility and exists as a new game for the framework, utilizing three critical entities defined within Clembench. These include a Player representing the model being evaluated, a programmatic Player for keeping track of the play space, and a Game Master for logging actions, parsing responses, and passing dialogue between both players.

3 Methods

Game Design SimpleSnake is designed to be played by a single LLM against a programmatic player, and centers around five core rules:

1. The snake has no tail.
2. The snake cannot leave the defined grid space.
3. The snake moves in single grid-space moves.
4. The snake can only move in cardinal directions.
5. The player must control the snake using the format: $< move : [up|down|left|right] >$.

There are three variations of the game: a normal variant, a variant with added obstacles, and one focused on planning the snake's route before traversing the grid.

Each game begins with an initial prompt, which is fed to the model being evaluated. This prompt primes the model to respond in the correct format, and gives it the rules that must be followed for the game to be completed successfully. An initial grid is then retrieved from the programmatic player, which is passed to the model for inference. After prompting the model with the grid, the model's response is parsed for correctness. If the model responds using an incorrect format, the game is aborted. Otherwise, the response is then given back to the programmatic player which updates the grid. Depending on which variation is being run, the programmatic player either informs the Game Master if the game was won or lost, or reports the game state back to the Game Master to continue

prompting the model. This latter cycle repeats until either the snake is successfully navigated to the food, the snake collides with something causing a 'game over' state, or a maximum of 20 moves is exceeded.

In total, 10 games are run for each experiment of a variation of SimpleSnake, with experiments being performed once for three different grid sizes. The three game variations implemented and used for evaluation are: SimpleSnake, SimpleSnake with Obstacles, and SimpleSnake with Planning.

Example Dialogue Figure 1 shows an example of SimpleSnake dialogue for an experiment on a 5x5 grid under the obstacle variant. On the left-hand side in orange are the responses from the model being evaluated, and on the right-hand side in purple are responses from the programmatic player. The Game Master is represented in the middle with gray text-bubbles. More dialogue examples can be found under appendix B, showing other game configurations.

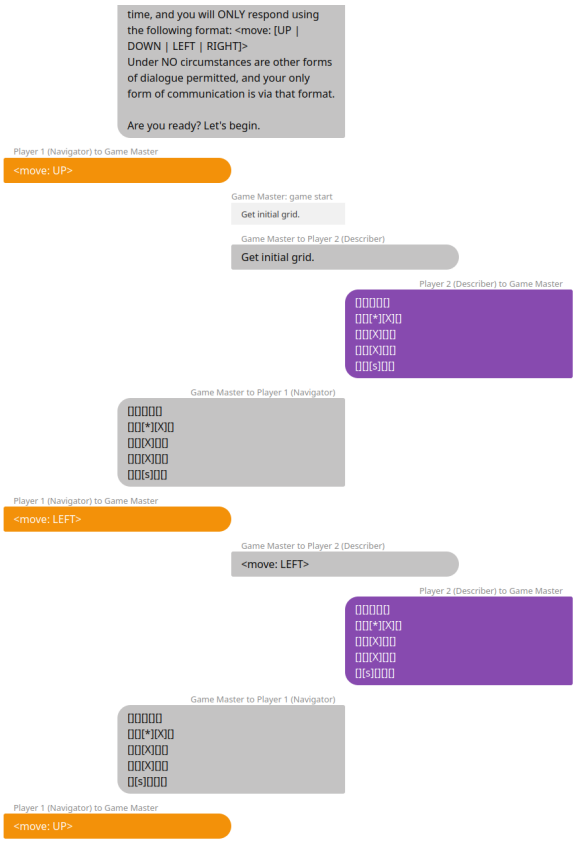

Figure 1: Example dialogue from a game of SimpleSnake, with the LLM being tested on the left in orange, and the programmatic player on the right in purple.

4 Experiments

4.1 Experiment 1: SimpleSnake

Task & Procedure The base variation of SimpleSnake follows a simple set of rules. It is played on a square text-grid, with two entities existing in the grid space. These are the snake and food, represented by 's' and '*' respectively. The model is tasked with navigating the snake one step at a time to the grid cell containing the food, without running into any grid boundaries and in fewer than 20 moves. This is done 10 times each on grids of size 5x5, 7x7, and 9x9.

Metrics There are three main metrics in any game within the Clembench framework. These are % Played, Quality Score, and the standard deviation of the Quality Score. In the case of SimpleSnake, % Played is purely based on the model's ability to adhere to responding in the correct format of $< move : [up|down|left|right] >$. Quality Score on the other hand measures the win rate of the model for games that are not terminated due to incorrect response formatting. These scores are compared against metrics for similar models on the AdventureGame and ReferenceGame tasks implemented within the Clembench framework as a baseline, which are seen in Figures 6 and 7 under appendix A.

Results Three different state-of-the-art models were evaluated on the normal version of SimpleSnake. Claude 3.7 Sonnet, GPT4o, and Gemini 1.5 Flash. As seen in Figure 2, all three models have no issue adhering to the correct response format and post scores of 100% for % Played. However, things differ when examining Quality Score. We see that Gemini 1.5 Flash struggles significantly to win games, only winning around one out of every four. In stark contrast, Claude 3.7 Sonnet is able to successfully play every game out of the 30 tested. GPT4o also does extremely well at the base version of SimpleSnake, although it does not have a perfect win rate like 3.7 Sonnet. Compared to the baseline metrics from other games, these scores indicate that for larger models like Claude 3.7 Sonnet and GPT-4o, the normal version of SimpleSnake is easier to execute than some existing Clembench games.

SimpleSnake	% Played	Quality Score	Quality Score (std)
Claude-3-7-Sonnet	100.00	100.00	0.00
GPT-4o	100.00	96.67	18.26
Gemini-1.5-Flash-002	100.00	26.67	44.98

Figure 2: Results from Experiment 1: SimpleSnake

4.2 Experiment 2: SimpleSnake with Obstacles

Task & Procedure Similar to the first experiment, the second experiment follows the same rules and is also played on square text-grids of size 5x5, 7x7, and 9x9. Unlike the first experiment, however, experiment 2 implements SimpleSnake with an additional game rule: obstacles spawn in the center part of the grid, and if the snake collides with an obstacle the game ends.

The purpose of this experiment is to get a better idea of what the model is doing when trying to complete the goal. Experiment 1 leaves open the possibility that the model is following a simple strategy of 'moving in the generally correct direction' instead of having a precise idea of the direction it is headed in. Obstacles are added to assess how much a model knows about its travel path or trajectory to the goal, and if it is capable of perceiving obstructions and navigating around them.

Metrics Experiment 2 uses the same metrics as those implemented in experiment 1, where % Played represents the number of games for which the model does not violate the required response format and Quality Score indicates the win rate of the model; the only difference between this variation of SimpleSnake and its normal game being that the model now needs to operate around obstacles. Because of this similarity, it makes sense for the metrics to be identical. The recorded results are compared against the previously mentioned baseline game metrics in Figures 6 and 7, under appendix A.

Results Analyzing and scoring the played games, it is immediately obvious that this task poses a noticeably more difficult challenge to the three models being evaluated on. Claude 3.7 Sonnet again performs the best on this metric out of those tested, although it is no longer able to win every game. Instead, there is about a 25% loss rate out of all 30 games played, with high standard deviation. GPT-4o shows the highest standard deviation, however, and drops down to a 40% win rate. Gemini 1.5 Flash does not lose much from its previous performance score, but this could be indicative of a high 'accidental' win rate from lucky food spawns. Compared to baseline, we can conclude that the obstacle variant of SimpleSnake is more difficult to execute than other comparable games in the framework. Figure 3 shows the metrics gathered for experiment 2, evaluated on Claude 3.7 Sonnet, GPT-4o, and Gemini 1.5 Flash.

w/ Obstacles	% Played	Quality Score	Quality Score (std)
Claude-3-7-Sonnet	100.00	76.67	43.02
GPT-4o	100.00	40.00	49.83
Gemini-1.5-Flash-002	100.00	23.33	43.02

Figure 3: Results from Experiment 2: SimpleSnake with Obstacles

4.3 Experiment 3: SimpleSnake with Planning

Task & Procedure Experiment 3 modifies the original game protocol, changing the flow of the dialogue significantly. Rather than going back and forth for some number of turns, the planning variant of SimpleSnake presented in this experiment requires that all moves needed to successfully navigate to the food are given up front at the beginning of each game. The main reason for doing this is to promote more precise planning strategies over alternative last-minute decision making, which the previous two experiments do not evaluate against.

Metrics Experiment 3 still uses the same metrics as both prior experiments, but the meaning of these metrics changes slightly along with the modified game rules. Because experiment 3 requires all moves to be given up front, the response format allowed for the model is expanded to be a sequence of directions given in the format $< move : [up|down|left|right] >$. The model still is not allowed to respond with anything but these directions, meaning that any deviations subtract from the % Played metric. As a result, the Quality Score now represents the number of games where the model responded using the correct for-

mat and was able to correctly provide a winning sequence of moves at the game's onset. Once again, the collected results are compared to the baseline metrics as found under appendix A.

Results The results from experiment 3 are presented in Figure 4, again showing the same three models being used for evaluation as with the previous experiments. Claude 3.7 Sonnet outperforms both other models to a high degree, gaining 10% to its win rate over the results from experiment 2. Interestingly, GPT-4o and Gemini 1.5 Flash both experience a noticeable degrade in performance on this new challenge, with their respective Quality Scores both dropping by around 50%. Across the board, standard deviations went down for all three models, indicating more stable results comparatively. Its worth noting that even with GPT-4o and Gemini 1.5 Flash seeing significant drops in performance, they are still able to abide by the proper format even while losing most of the games they played. Compared to baseline, it is visible that the performance curve for models on this variation is much higher than that of other games, leaving more room for upward leaderboard movement.

w/ Planning	% Played	Quality Score	Quality Score (std)
Claude-3-7-Sonnet	100.00	86.67	34.57
GPT-4o	100.00	26.67	44.98
Gemini-1.5-Flash-002	100.00	10.00	30.51

Figure 4: Results from Experiment 3: SimpleSnake with Planning

5 Conclusion

5.1 Implications

The cumulative results shown in Figure 5 and those gathered in section 4 show that SimpleSnake exists as a more challenging set of tasks compared to similar games already within the Clembench framework, with a steeper performance curve as models progress through its sub-games and experiments. At the same time, because % Played remains at 100% for all of the tested models, we can extrapolate that the game itself is highly tractable. This is important because it means that the game is not excluding models based on incorrect formatting, but instead scoring more on a model's ability to actually internalize the game rules and perform each task. That being said, although SimpleSnake is highly tractable, it is not trivial to perform well on. The Avg. Quality Scores in Figure 5 imply that only incredibly large models with superior logical reasoning are capable of posting scores comparable to other games, with experiment 3 further implying that most current models are not able to consistently perform well at planning routes in 2D text-based spaces.

Model	clemscore	Avg. % Played	Avg. Quality Score
Claude-3-7-Sonnet	87.78	100.00	87.78
GPT-4o	54.45	100.00	54.45
Gemini-1.5-Flash-002	20.00	100.00	20.00

Figure 5: Clembench score averaged across variations.

5.2 Limitations

There are a number of limitations present within this work that should not go ignored. The first is the nature of the initial game states used for each experiment. Although each model is evaluated under exactly the same circumstances, these game states were created using random generation. While more in-line with how a real game of snake would typically take place, there are problems with this approach. Namely, that the resulting game boards can circumvent the intended purpose of the obstacle and planning variants of SimpleSnake. For example, obstacles are occasionally generated in a way that places them outside of the direct starting path from the snake to its food, in some ways reducing that game to another instance of the normal variation. In a similar fashion, it is also possible for the snake and the food to be spawned adjacent to each other. For the planning variant, this results in a game that could take place in the normal variation, and completely circumvents the intentions of forcing a model to plan more than one move in advance.

Another limitation of the current system is the number of obstacles being generated for each grid, and where they can be generated. Although small, there is a chance when randomly generating obstacles to end up with a non-winnable state if the number of generated obstacles is greater than three, or in some cases greater than two. This would occur if obstacles are generated in all grid cells

directly adjacent to the food, one for each available cardinal direction. To avoid these edge cases, the game currently generates a maximum of three obstacles per grid, and will not generate them on the edges of the board. This is potentially problematic, particularly as the size of the grid increases, which begins to decrease the occurrences of obstacles in the direct path between the snake and the food as obstacles become more sparse within the grid.

The potentially varying quality of initial grid states in the experiments currently present in SimpleSnake leave room for unwanted inconsistency in model evaluation. All of these problems should be addressable with more research time, outlined in the next section.

5.3 Future Work

The results of this project shine light on multiple avenues for future work, including improvements that can be made to the SimpleSnake task and additional consequential works.

Going forward, there are multiple revisions that can be made to the current version of SimpleSnake that might improve results and serve to better evaluate models. Potential directions include:

- Altering the instance generation logic to ensure that the snake and food spawn at an optimal distance from each other. This would increase the bar for a model's performance, resulting in fewer unearned Quality Score points.

- Adding checks to entity positioning during instance generation to prevent non-winnable edge cases and allowing for obstacles to spawn in greater numbers and across the entire grid space. This would reduce the problem of obstacle sparseness as gird size increases, and would more closely align the obstacle avoidance task with its intended purpose.

- Experimenting with different scoring metrics that reward partial completeness instead of pure win-rate, focusing on individual decision quality and movement towards the food at game end if the game is lost.

- Investing time into collecting human-created or human-curated game boards for all variations, instead of relying on random generation.

Outside of future work on SimpleSnake itself, this project highlights a need for future investigations into the usage of LLMs for navigating spaces. Particularly those beyond 2D text-based spaces, which could result in the consolidation of model footprints in applications like embodied artificial intelligence. Foremost, I see potential for investigating the use of LLMs in navigating through 3D spaces. This research might take place as evaluating a model's ability to understand and act in 3D space, or giving an LLM a portrayal of 3D space in two dimensions to navigate around physical objects in a robot form factor.

References

Kranti Chalamalasetti, Jana Götze, Sherzod Hakimov, Brielen Madureira, Philipp Sadler, and David Schlangen. 2023. Clembench: Using game play to evaluate chat-optimized language models as conversational agents.

Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. 2024. Chatbot arena: An open platform for evaluating llms by human preference.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need.

A Appendix – Baseline Metrics

Figures 6 and 7 show official scores for Claude 3.7 Sonnet, GPT-4o, and Gemini 2.0 Flash Experimental on AdventureGame and ReferenceGame respectively. These scores are taken directly from the Clembench leaderboard [3].

AdventureGame	% Played	Quality Score	Quality Score (std)
Claude-3-7-Sonnet	96.88	84.90	33.43
GPT-4o	94.53	71.61	36.96
Gemini-2.0-Flash-Exp	67.97	50.52	44.66

Figure 6: AdventureGame Baseline

[3] https://clembench.github.io/leaderboard.html

ReferenceGame	% Played	Quality Score	Quality Score (std)
Claude-3-7-Sonnet	100.00	98.89	10.54
GPT-4o	100.00	88.89	31.60
Gemini-2.0-Flash-Exp	100.00	82.22	38.45

Figure 7: ReferenceGame Baseline

B Appendix – Example Dialogue

The following are examples of dialogue from SimpleSnake games, with different variants. The left-hand side in orange are responses from the model being evaluated, and the responses on the right-hand side in purple are from the programmatic player as implemented in SimpleSnake. The Game Master is represented in the middle by gray text-bubbles.

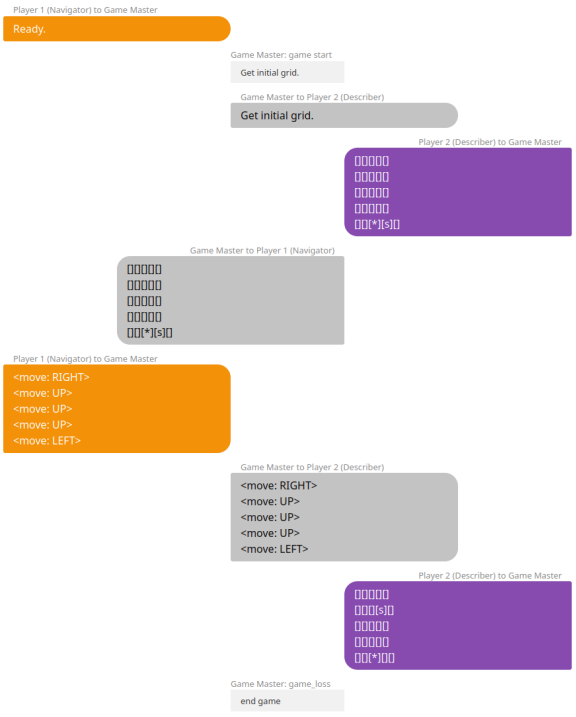

Figure 8: Example dialogue for a 5x5 game of SimpleSnake with planning.

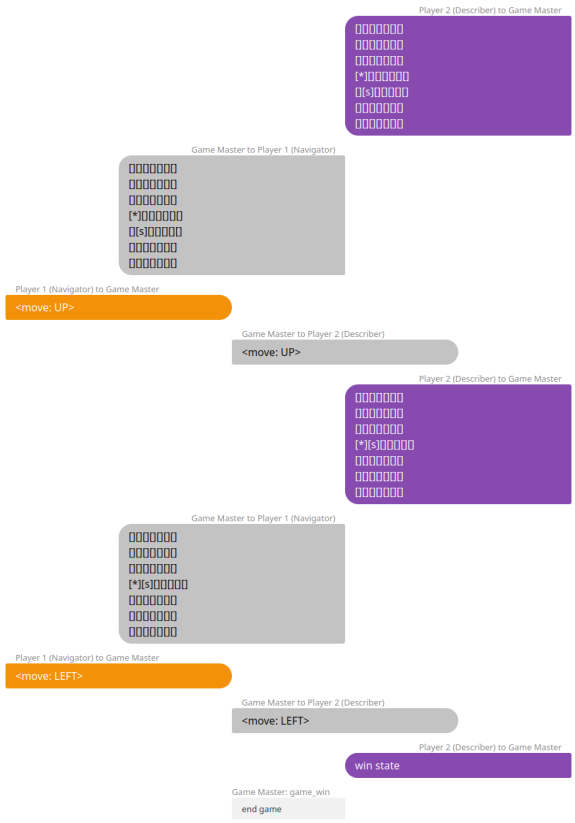

Figure 9: Example dialogue for a normal 7x7 game of SimpleSnake.

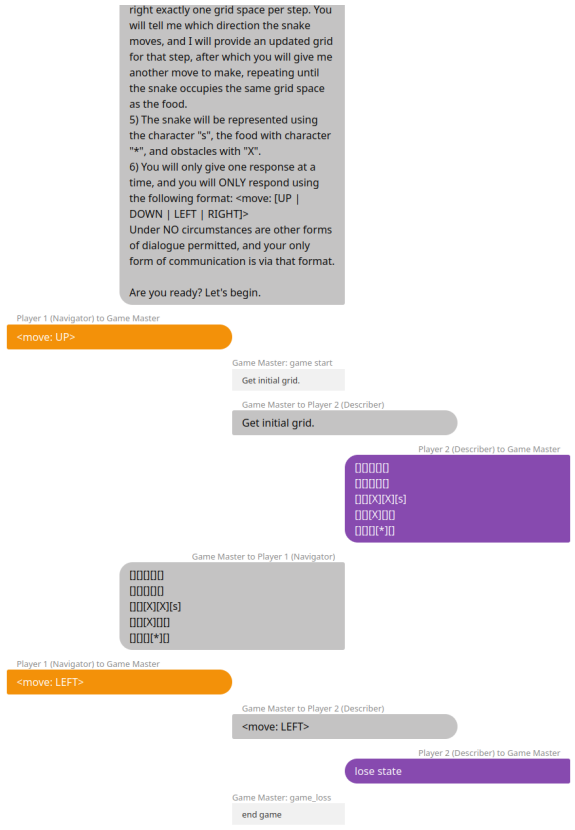

Figure 10: Example dialogue for a 5x5 game of SimpleSnake with obstacles.